

BACKEND LEARNING JOURNEY

DAY 14 • PROJECT 2 — URL SHORTENER

Node.js + Express + PostgreSQL

Design Requirements • API Design • URL Model • Short-Code Strategy

Today's goal

Understand the complete design of the URL shortener before implementation. The project follows the learning loop: **Understand** → **Implement** → **Test** → **Break** → **Fix** → **Explain** → **Reuse**.

Day 14 is the design stage. Day 15 begins implementation of short URL creation.

1. What Is a URL Shortener?

A URL shortener converts a long URL into a short, easy-to-share URL. The short URL contains a short code that the backend uses to find the original URL.

Example

Long URL:

`https://example.com/articles/backend-development/how-to-build-a-rest-api`

Short URL:

`https://short.ly/aB72xK`

Core mapping

`shortCode` → `originalUrl`

`aB72xK` → `https://example.com/very-long-url`

2. Main Operations

Operation	HTTP	Purpose
Create short URL	POST /shorten	Create a new short URL
Redirect	GET /:shortCode	Find the original URL and redirect

3. Create Short URL — API Design

Endpoint

POST /shorten

Request body

```
{
  "url": "https://example.com/very-long-url"
}
```

Backend flow

1	Receive URL
2	Validate URL
3	Generate random shortCode
4	Check whether the shortCode already exists
5	If collision → generate another code
6	Save originalUrl + shortCode
7	Return response

Example response

```
{
  "originalUrl": "https://example.com/very-long-url",
  "shortCode": "aB72xK",
  "shortUrl": "https://short.ly/aB72xK"
}
```

4. Short-Code Strategy

Our backend generates a **random 6-character alphanumeric short code**.

Character set	a–z, A–Z, 0–9
Number of characters	6
Possible combinations	$62^6 = 56,800,235,584$
Generated by	Backend
Uniqueness	Database check + UNIQUE constraint

Examples: aB72xK kLm9pQ Xy7H2n z8QwEr

5. Collision Handling

Because the short code is random, the backend can theoretically generate a code that already exists. This is called a **collision**.

Collision-safe flow

```
Generate shortCode
↓
Check database
↓
Exists? – YES → Generate another code
Exists? – NO → Save it
```

We will also enforce uniqueness at the database level with a **UNIQUE constraint** on shortCode.

6. Initial URL Database Model

Field	Purpose
id	Unique database record ID
originalUrl	The original long URL
shortCode	Generated short identifier

Example record

```
id: 1
originalUrl: https://example.com/very-long-url
shortCode: aB72xK
```

A user ID is not required for our basic version because URL ownership is not part of the initial requirements.

7. Redirect Flow

Endpoint

```
GET /:shortCode
```

Example: GET /aB72xK

The backend extracts **aB72xK**, searches the database, gets the originalUrl, and redirects the browser to that URL.

```

GET /aB72xK
↓
Find aB72xK in database
↓
Get originalUrl
↓
Redirect to original URL

```

8. Complete System Architecture

CLIENT / BROWSER	→	NODE.JS + EXPRESS	→	POSTGRESQL
Send request		Validate • Generate • Check • Save • Redirect		Store URL mapping

Create flow:

Client → POST /shorten → Node.js/Express → Generate code → Check collision → PostgreSQL → Response

Redirect flow:

Browser → GET /:shortCode → Node.js/Express → PostgreSQL → originalUrl → Redirect

9. What We Are NOT Adding Yet

Day	Feature
18	Analytics — click count, timestamps, basic statistics
19	Expiration — expiry rules and expired-link behavior
20	Rate limiting
21	Testing — happy paths, invalid URLs, collisions, expired links
22	Independent challenge

10. Day 14 — What You Should Remember

1. A URL shortener maps shortCode → originalUrl.
2. POST /shorten creates a short URL.
3. The backend generates the short code.
4. Our strategy is a random 6-character alphanumeric code.
5. Random codes can collide, so we check the database and retry.
6. shortCode should have a UNIQUE constraint.
7. The initial model needs id, originalUrl, and shortCode.
8. GET /:shortCode finds the original URL and redirects the user.
9. We add analytics, expiration, rate limiting, and deeper testing on later days.

11. Day 14 Completion

Task	Status
Understand URL Shortener	✓ Complete

Requirements	✓ Complete
API design	✓ Complete
URL model	✓ Complete
Short-code strategy	✓ Complete
Collision concept	✓ Complete
Redirect flow	✓ Complete
System architecture	✓ Complete
Implementation	→ Day 15

Next: Day 15 — Create Short URLs: POST endpoint, validation, and unique code generation.